

Arranque + planificación preliminar



Alejandro Peña Fernández

Índice:

RESUMEN:	3
Sistema de gestión para comunidades de vecinos:	3
Acceso al sistema:	3
Módulo de presupuesto:	3
Fondo de emergencia:	3
Módulo de gastos:	3
Tablón de votaciones:	4
Tecnologías:	4
DESCRIPCIÓN DETALLADA:	4
COMPARTIDOS:	4
NAVBAR:	4
FOOTER:	4
ESPECÍFICOS:	5
INICIO:	5
PRESUPUESTOS:	5
GASTOS:	5
VOTACIÓN:	5
OBJETIVOS:	6
Objetivo principal:	6
Objetivos específicos:	6
- Qué entra (alcance del MVP):	6
- Qué no entra (límites):	7
- Soluciones a posibles problemas:	7
CONTENIDOS DEL CICLO:	7
Desarrollo web en entorno servidor (DWES):	7
Desarrollo web en entorno cliente (DWECE):	7
Diseño de interfaces web (DIW):	7
PLANIFICACIÓN PRELIMINAR:	8
- Sprint 1:	8
- Sprint 2:	8
- Sprint 3:	8
- Sprint 4:	8
- Sprint 5:	9
- Sprint 6:	9
- Sprint 7:	9
- Sprint 8:	9
- Sprint 9:	9
ENTORNO DE TRABAJO:	9

RESUMEN:

Sistema de gestión para comunidades de vecinos:

Software diseñado para facilitar la administración de comunidades de vecinos, permitiendo al presidente gestionar presupuestos, gastos y decisiones comunitarias, mientras que los vecinos pueden consultar la información de manera transparente y ejercer su derecho a voto. El objetivo es agilizar los procesos administrativos y optimizar el tiempo de todos los miembros de la comunidad.

Acceso al sistema:

El acceso a la plataforma se realizará mediante un sistema de inicio de sesión. El registro de nuevos usuarios será responsabilidad del presidente de la comunidad cuando se incorpore un nuevo vecino al edificio.

Módulo de presupuesto:

Esta sección mostrará el presupuesto anual acordado por la comunidad, el detalle de gastos ejecutados y el saldo disponible tras descontar los gastos actuales. El presidente será responsable de registrar el presupuesto anual y los gastos mensuales.

Fondo de emergencia:

Se habilitará una pestaña específica donde se visualizará el fondo de emergencia, que vendrá del dinero sobrante al finalizar cada año.

Módulo de gastos:

Esta sección presentará una lista detallada con los gastos fijos mensuales y los gastos variables del mes en curso. El presidente podrá añadir nuevos gastos mediante un botón específico. Por defecto, los gastos se descontarán del presupuesto anual.

Tablón de votaciones:

Se dispondrá de un área donde se mostrarán las votaciones en curso y aquellas finalizadas en el último mes. La información se presentará en forma de lista con el título de cada votación y un botón para acceder a información detallada.

Para votaciones en curso donde el usuario no haya participado, se abrirá una ventana que permitirá votar a favor, en contra o abstenerse. Una vez emitido el voto, estos botones quedarán bloqueados.

En el caso de votaciones finalizadas, se mostrará la misma información, pero en lugar de los botones de votación aparecerá el resultado: aprobada o denegada.

Las votaciones concluirán cuando todos los vecinos hayan ejercido su voto o al alcanzarse la fecha límite establecida.

Tecnologías:

Las tecnologías a utilizar son HTML, CSS, Javascript, React, Java, SpringBoot y PostgreSQL.

DESCRIPCIÓN DETALLADA:

COMPARTIDOS:

NAVBAR:

Tendrá presupuestos, gastos y votaciones; que llevará respectivamente a sus páginas correspondientes.

FOOTER:

Tendrá el nombre del bloque con el símbolo de copyright y el año actual. Excepto en la página de inicio, tendrá un botón volver que llevará a la página de inicio.

ESPECÍFICOS:

INICIO:

Aparecerá una foto del bloque o algún fondo chulo y la bienvenida al edificio concreto.

PRESUPUESTOS:

Tendrá el gastos pactado anual, los gastos anuales que llevan hechos y el dinero actual que quede. Estos valores por defecto vendrán en 0 hasta que el presidente los cambie, para ello habrá un botón editar que llevará a un formulario en el que se pondrá el presupuesto anual y los gastos fijos de ese año con su respectivo nombre y descripción. El gastos que se lleva y el dinero actual será un cálculo del presupuesto anual junto con los gastos mensuales.

También se verá una lista de los gastos fijos anuales con los gastos en años y meses.

Abajo del todo se verá el fondo de emergencia que será el dinero guardado anualmente del sobrante del presupuesto anual.

GASTOS:

Se diferenciarán dos tarjetas, una con una lista de gastos fijos y otra con una lista de gastos imprevistos mensuales.

VOTACIÓN:

Una página con una lista de votaciones tanto pasadas como las presentes con la opción de entrar en cada una.

En las votaciones verás la información del motivo de la votación y podrás votar a favor, en contra o abstenerse, una vez votado no podrás volver a votar.

En votaciones acabas verás la información de la votación y cuál fue el resultado.

OBJETIVOS:

Objetivo principal:

Crear una aplicación web para que las comunidades de vecinos puedan gestionar su día a día sin tener que depender de papeles, grupos de WhatsApp o reuniones eternas. La idea es que el presidente pueda llevar los números y las votaciones desde la plataforma, y que los vecinos puedan ver en qué se gasta el dinero y votar cuando toque.

Objetivos específicos:

Sistema de acceso con roles: Que cada usuario entre con su cuenta y vea lo que le corresponde. El presidente puede hacer cosas que un vecino normal no (como registrar gastos o crear votaciones). Los vecinos nuevos los da de alta el presidente.

Gestión de presupuestos: Que el presidente pueda meter el presupuesto anual y los gastos fijos del año. Que se calcule automáticamente cuánto se lleva gastado y cuánto queda. Que se vea el fondo de emergencia con lo que sobre de cada año.

Control de gastos: Que se puedan ver los gastos fijos y los imprevistos del mes por separado. Que el presidente pueda añadir gastos nuevos y que se descuenten solos del presupuesto.

Tablón de votaciones: Que los vecinos puedan votar a favor, en contra o abstenerse en las votaciones abiertas. Que una vez votes no puedas cambiar el voto. Que las votaciones se cierren cuando todos hayan votado o cuando se pase la fecha límite. Que se vea el resultado de las votaciones que ya han acabado.

- Qué entra (alcance del MVP):

Un solo edificio/comunidad por instalación.

Dos roles: presidente y vecino.

CRUD de presupuestos, gastos y votaciones.

Autenticación con usuario y contraseña.

Consulta de información financiera por parte de los vecinos.

Votaciones con tres opciones (a favor, en contra, abstención).

- Qué no entra (límites):

No se gestionan cobros reales ni pasarelas de pago. Esto solo refleja números, no mueve dinero.

No hay chat ni mensajería entre vecinos.

No se gestiona la morosidad ni los recibos individuales de cada vecino.

No hay notificaciones por email ni push de momento.

No se contempla que haya varios edificios en la misma plataforma (multitenancy).

No se implementa recuperación de contraseña en el MVP.

No hay subida de documentos (actas, facturas escaneadas...).

- Soluciones a posibles problemas:

En un futuro se ampliará un apartado en el que se cometen diferentes soluciones o propuestas a problemas legales relacionados con la normativa española.

CONTENIDOS DEL CICLO:

Desarrollo web en entorno servidor (DWES):

Java y Spring Boot para el backend creando una API Rest para la lógica de negocio, autenticar al usuario con sus diferentes roles...

Desarrollo web en entorno cliente (DWECE):

Javascript y React para la interfaz visual creando componentes para la votación, visualización de presupuestos, navegar entre pestañas...

Diseño de interfaces web (DIW):

CSS para la estética visual de la aplicación web diseñando una interfaz sencilla para el usuario, con armonía y que entre por los ojos.

Despliegue de aplicaciones web:

Se usará algún proveedor, todavía no definido, para que se despliegue la aplicación y se conecten las diferentes partes.

PLANIFICACIÓN PRELIMINAR:

- Sprint 1:

Definir mi diseño (Wireframes) -> 45 minutos

Crear un prototipo -> 20 minutos

Investigar tecnologías -> 40 minutos

Definir funcionalidad/tecnologías -> 20 minutos

Investigar tecnologías -> 1 hora

Preparar infraestructura (Docker, BD...) -> 50 minutos

Implementar estructura (Carpetas, Git...) -> 40 minutos

Crear diseño BD y definir entidades -> 40 minutos

Crear endpoints -> 20 minutos

Crear entidades y relaciones de la base de datos -> 1 hora y 30 minutos

- Sprint 2:

Diseñar backend -> 2 horas

Crear las funcionalidades mínimas -> 2 horas y 45 minutos

Documentar el código -> 45 minutos

Estudiar nuevas funciones -> 1 hora 30 minutos

- Sprint 3:

Diseñar frontend -> 2 horas

Crear las funcionalidades mínimas -> 3 horas

Documentar el código -> 45 minutos

Estudiar nuevas funciones -> 1 hora y 30 minutos

- Sprint 4:

Conectar backend y frontend -> 3 horas

Arreglar posibles fallos -> 45 minutos

- Sprint 5:

Añadir funcionalidades al backend -> 7 horas

Documentar el código -> 1 hora

Estudiar nuevas funciones -> 2 horas

- Sprint 6:

Añadir funcionalidades al frontend -> 7 horas

Documentar el código -> 1 hora

Estudiar nuevas funciones -> 2 horas

- Sprint 7:

Mejorar diseño estético -> 1 hora y 30 minutos

Probar conexión y funcionalidades -> 2 horas y 30 minutos

Arreglar posibles fallos -> 2 horas y 30 minutos

- Sprint 8:

Desplegar frontend -> 45 minutos

Desplegar base de datos -> 45 minutos

Desplegar backend -> 2 horas

Solucionar problemas -> 30 minutos

- Sprint 9:

Últimas mejoras -> 2 horas y 30 minutos

Últimas pruebas -> 1 hora y media

Arreglar posibles fallos -> 2 horas

ENTORNO DE TRABAJO:

Para el desarrollo del proyecto utilizaremos una carpeta principal que contendrá todo el código y la documentación, organizada de la siguiente manera: frontend, backend, docs, docker (si aplica según decisiones a tomar más adelante), .gitignore y README.md.

La carpeta docs/ albergará la bitácora del proyecto, que por ahora se llevará en un documento Word con el seguimiento de decisiones y lecciones aprendidas a lo largo del desarrollo. Más adelante se decidirá el formato concreto de entrega.

El .gitignore estará configurado desde el inicio para excluir archivos generados automáticamente como node modules, los archivos de variables de entorno y otras configuraciones sensibles.

Para el control de versiones utilizaremos Git con tres ramas principales: main para el código estable, developer como rama de integración y feature/nombre-funcionalidad para el desarrollo de cada nueva funcionalidad. Además, contaremos con un repositorio remoto en GitHub.

Respecto al despliegue, el proyecto está pensado para funcionar sobre Vercel (frontend), Render (backend) y Neon (base de datos PostgreSQL), aunque también se contempla la opción de despliegue en una VPS o algún servicio con mejores prestaciones en el momento como alternativa.